Second Year Engineering

23CSE3004 Operating System

Class - S.Y. (SEM-II)

Unit - I

PROCESS MANAGEMENT

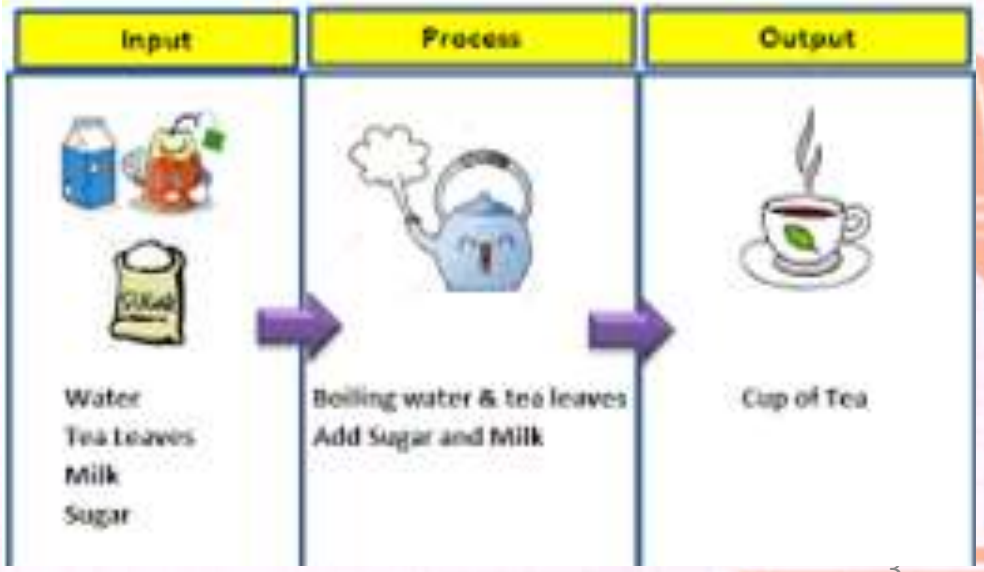
AY 2025-2026 SEM-II

Unit-I

Process Management: Process definition, the process concept – system programmer's view of processes – operating system's views of processes, States, Process Control Block (PCB), Process Creation and Termination (fork, exec, process hierarchy, exit, wait), Context switching, **Types of schedulers (long-term, short-term, medium-term), Scheduling algorithms- Preemptive vs. Non-Preemptive Scheduling (FCFS, SJF, RR, Priority Scheduling)**

Process

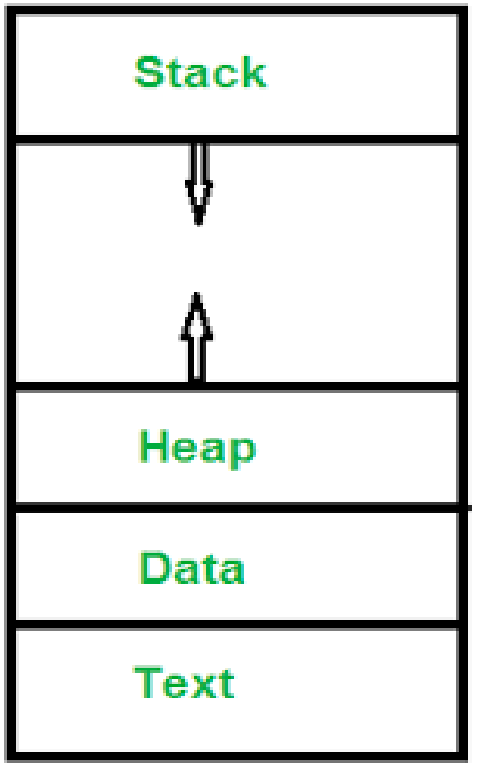
- Program under execution
- An instance of a program
- Schedulable/Dispatch unit (CPU)
- Unit of execution (CPU)



Process as a Data structure

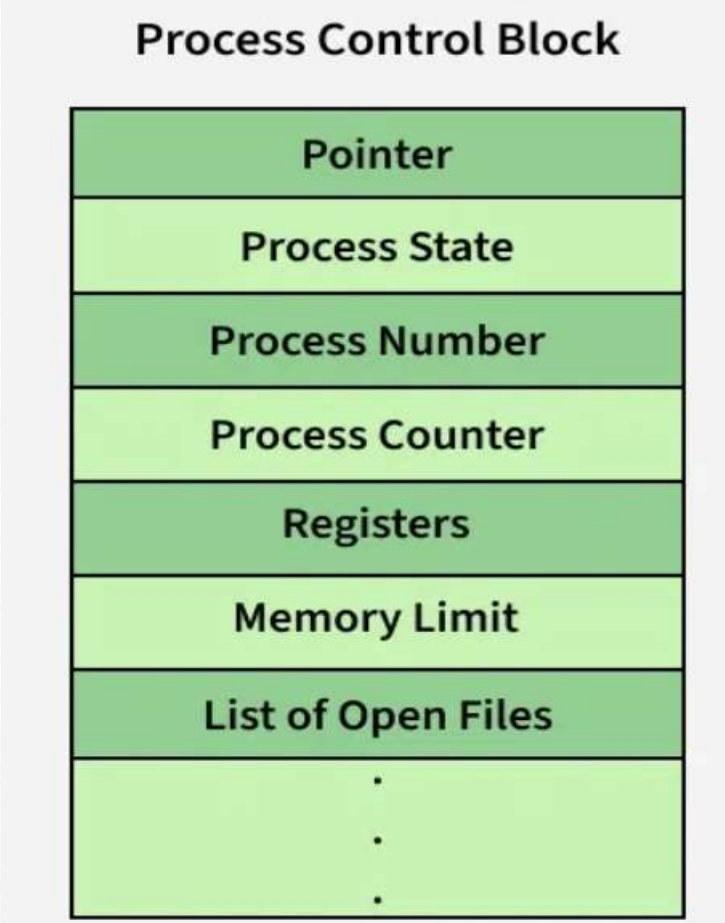
- Definition (Same as last slide)
- Representation/Implementation
- Operations
- Attributes

Process representation



Process Attributes in Process Control Block (PCB)

A Process Control Block (PCB) is a data structure used by the operating system to manage information about a process. The process control keeps track of many important pieces of information needed to manage processes efficiently.



- **Pointer:** It is a stack pointer that is required to be saved when the process is switched from one state to another to retain the current position of the process.
- **Process state:** It stores the respective state of the process.
- **Process number:** Every process is assigned a unique id known as process ID or PID which stores the process identifier.
- **Program counter:** stores the counter, which contains the address of the next instruction that is to be executed for the process.
- **Registers:** When a processes is running and it’s time slice expires, the current value of process specific registers would be stored in the PCB and the process would be swapped out. When the process is scheduled to be run, the register values is read from the PCB and written to the CPU registers.
- **Memory limits:** This field contains the information about memory management system used by the operating system.
- **List of Open files:** This information includes the list of files opened for a process.

- The **Process Table** is an array of PCBs, which logically contains a PCB for all of the current processes in the system.
- Adv: To keep track processes, schedule, easy process management
- Disadv: Memory slower operation extra work

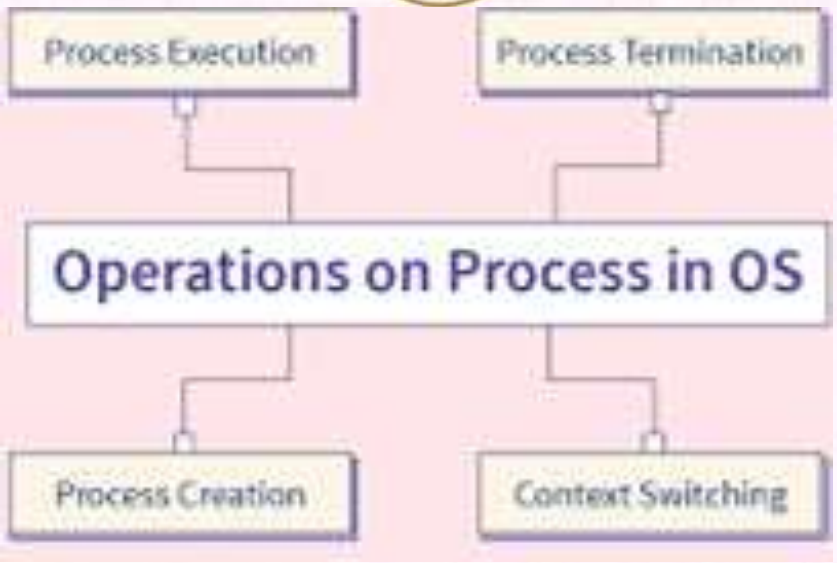
More about PCBs

- OS maintains 1 PCB for each process
- Contents of a PCB are also called collectively as context of a process

OS PCB1, PCB2, PCB3…PCBn
Process 1 (Stack, heap, code, data sections)
Process 2 (Stack, heap, code, data sections)
Process 3 (Stack, heap, code, data sections) . .
Process n (Stack, heap, code, data sections)

Process operations

- Create (space allocation)
- Schedule/run
- Wait/Block
- Suspend/Resume
- Terminate (space deallocation)



Multiprogramming

We have many processes ready to run. There are two types of multiprogramming:

- **Preemption:** Process is forcefully removed from CPU. Pre-emption is also called time sharing or multitasking.

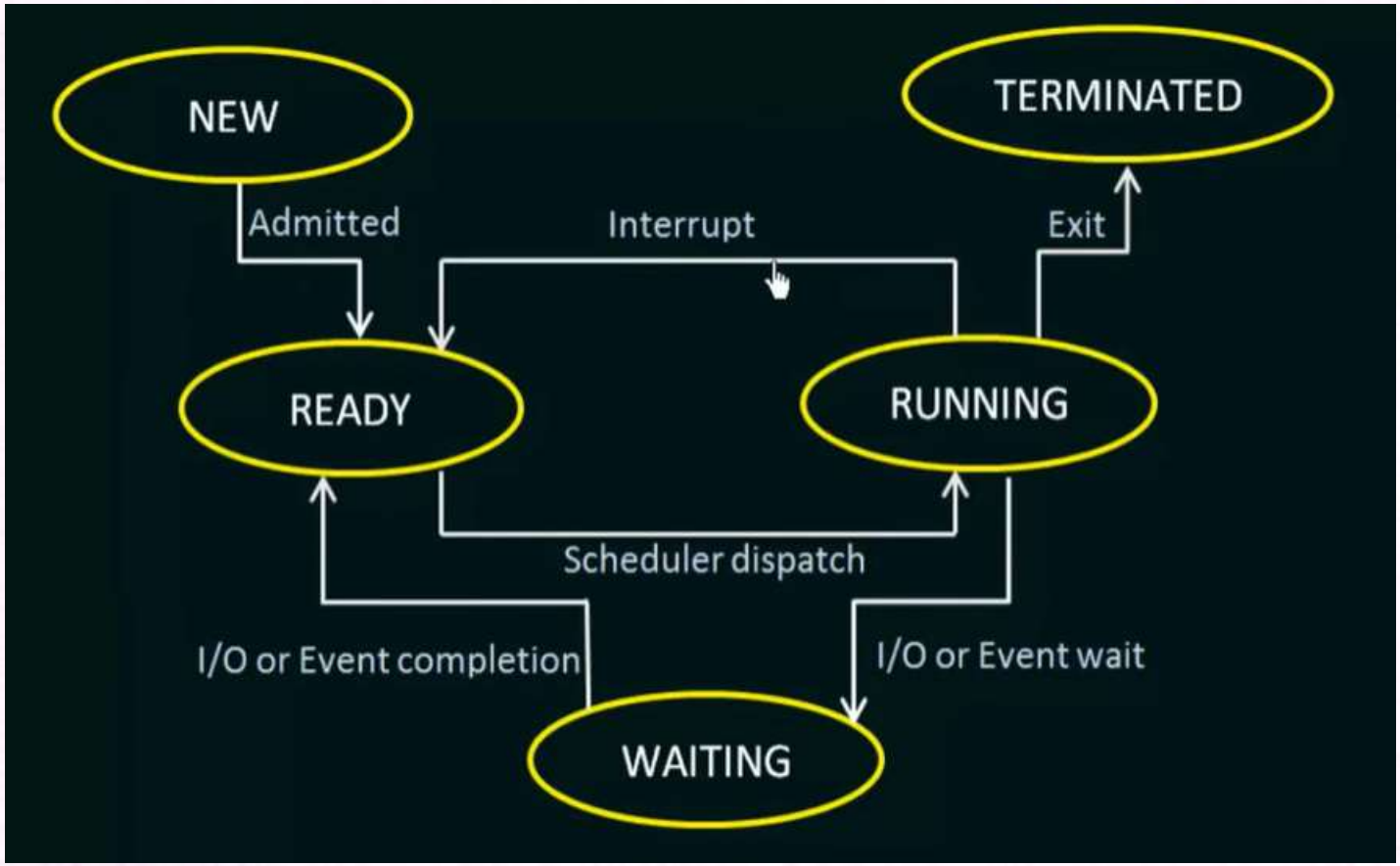
- **Non-Preemption:** Processes are not removed until they complete the execution. Once control is given to the CPU for a process execution, till the CPU releases the control by itself, control cannot be taken back forcibly from the CPU.

Process States

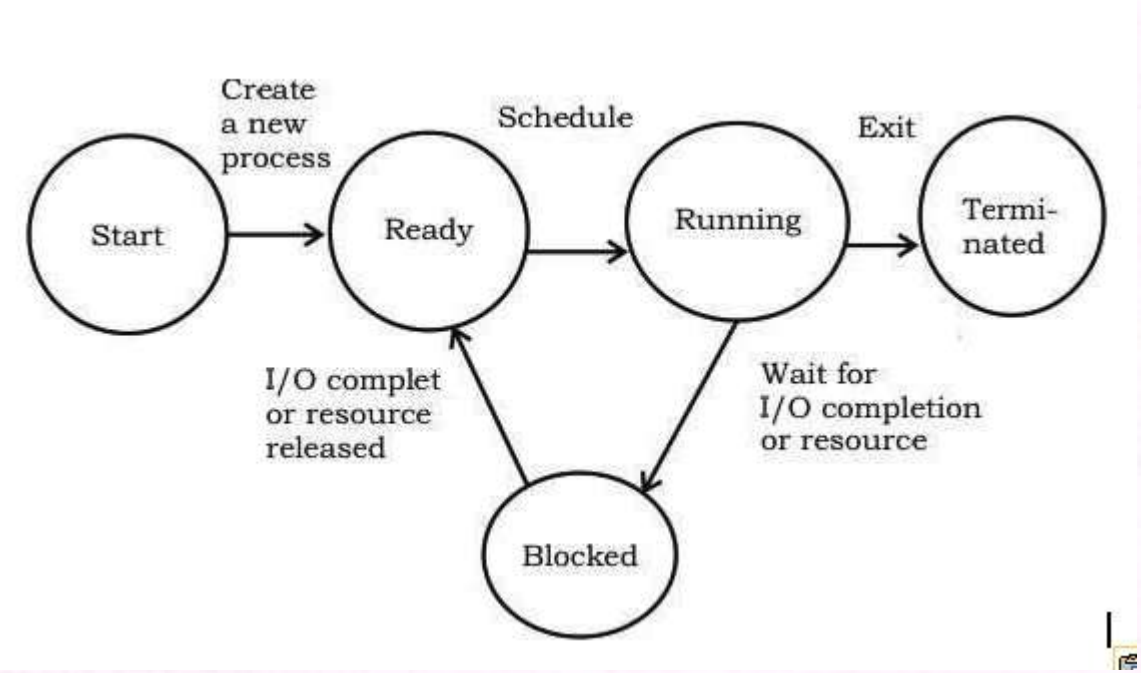
- In an operating system, a process is a program that is being executed. During its execution, a process goes through different states.

- Understanding these states helps us see how the operating system manages processes, ensuring that the computer runs efficiently.

Preemptive Process States



Non-preemptive process states

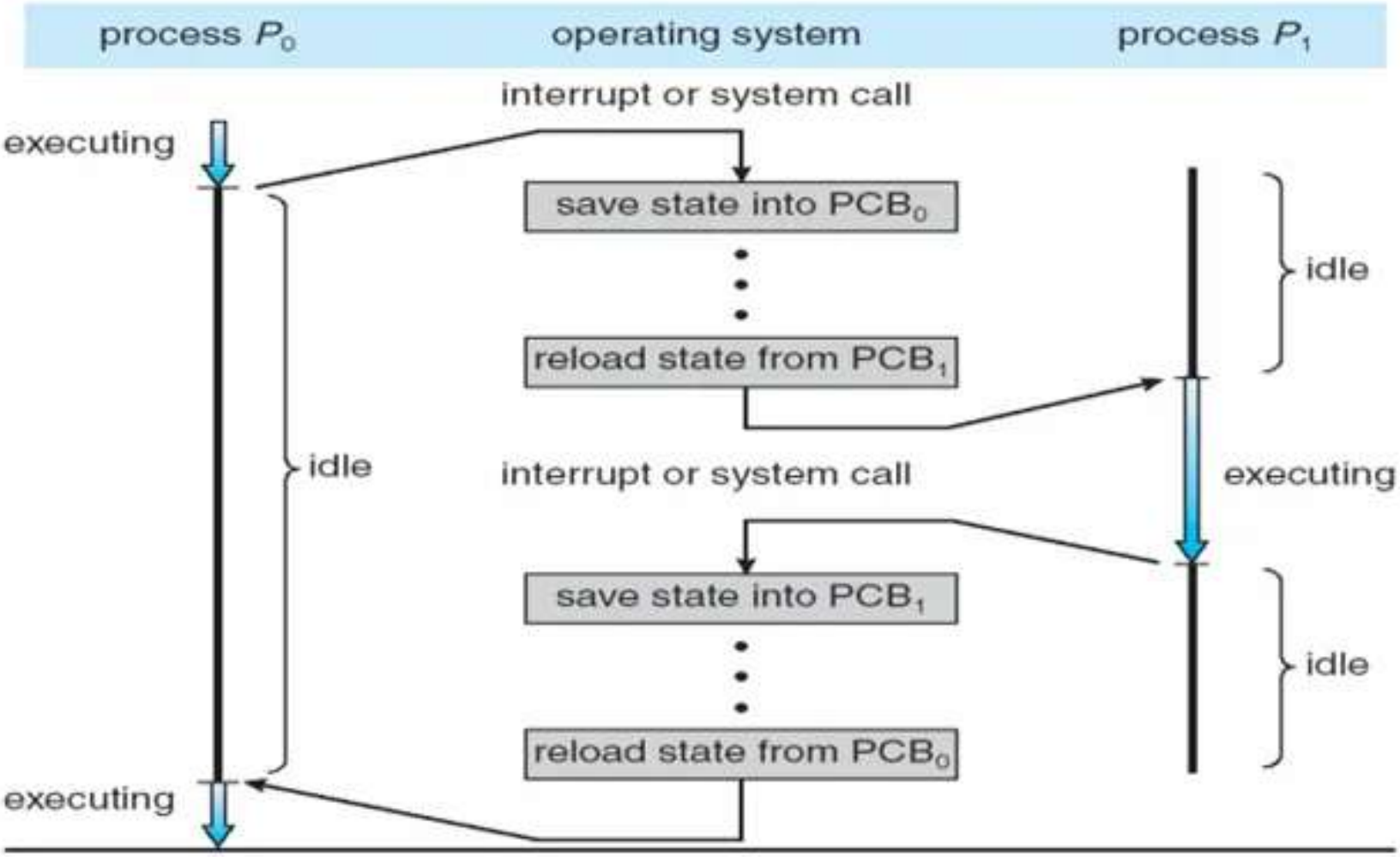


- **New:** This state represents a newly created process that hasn't started running yet. It has not been loaded into the main memory, but its process control block (PCB) has been created, which holds important information about the process.
- **Ready:** A process in this state is ready to run as soon as the CPU becomes available. It is waiting for the operating system to give it a chance to execute.
- **Running**: This state means the process is currently being executed by the CPU. Since we're assuming there is only one CPU, at any time, only one process can be in this state.
- **Blocked/Waiting:** This state means the process cannot continue executing right now. It is waiting for some event to happen, like the completion of an input/output operation (for example, reading data from a disk).
- **Exit/Terminate:** A process in this state has finished its execution or has been stopped by the user for some reason. At this point, it is released by the operating system and removed from memory.

Context Switching in OS

- CPU uses registers to store current process execution state
- When stopped in between, have to save these values in PCB to resume from that point onwards
- Saving current running process's content(CPU register values) from CPU to PCB of the process and loading other process's context in CPU registers

Example Context Switching



Note

- When a new process is formed, OS makes the PCB
- In non preemptive process, no factor can force a running process to come out of CPU. Termination or going to waiting is only its wish.
- PCB will be there in running, ready and waiting state. In these three states, process will be in MM only

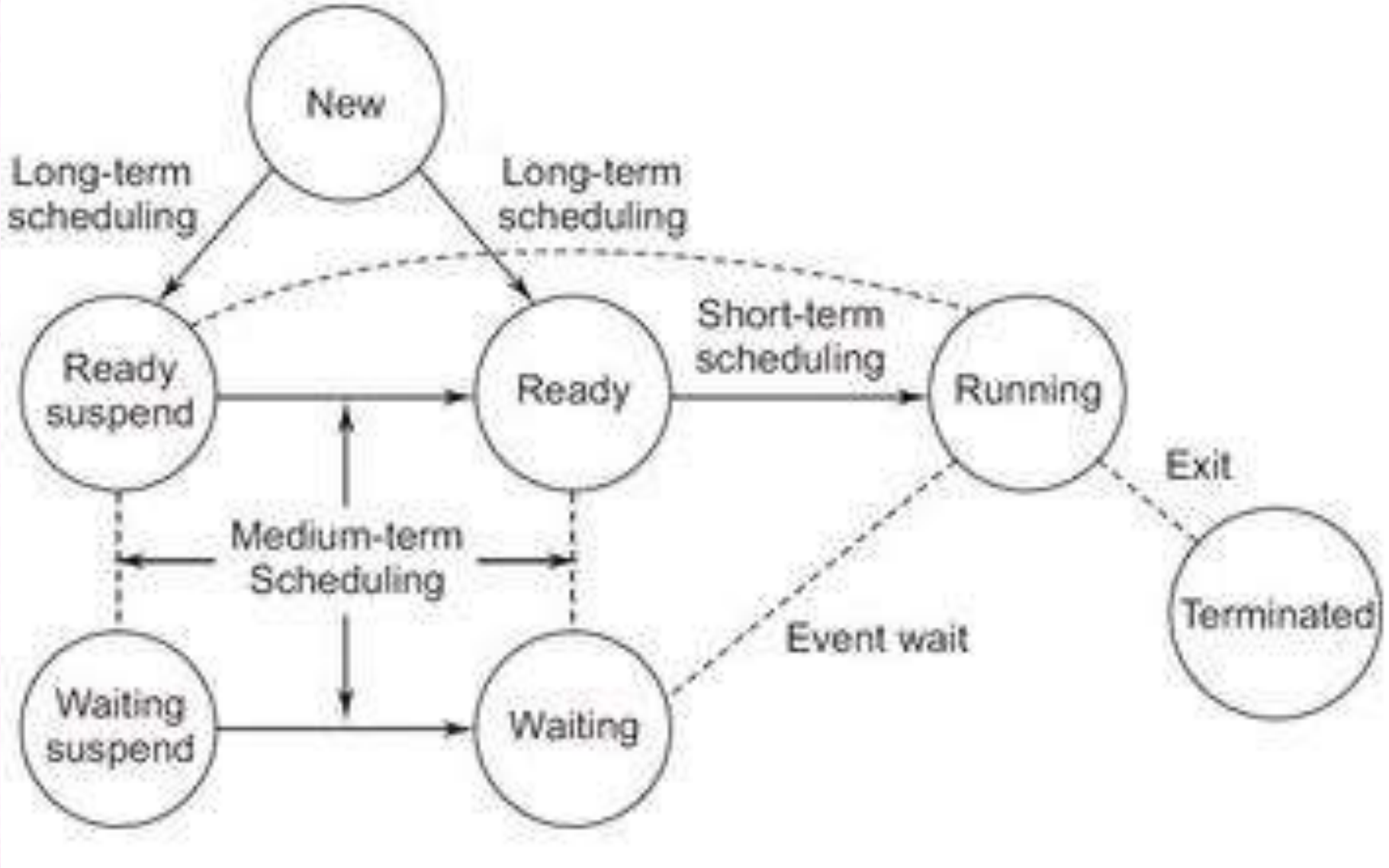
Process Scheduling

- Long term scheduler: new to ready
- Short term scheduler: ready to running
- Mid term scheduler: swapping is done by this. Swapping leads to suspension

- Job queue: to keep all processes in new state
- Ready queue: to keep all processes in ready state
- Device queue: to keep the processes in blocked state

CPU scheduling

- Make a selection of ready processes to run on CPU.
- Needed for better performance
- Metric for performance may vary
- Multiple policies according to metric


New
Long-term scheduling
Long-term scheduling
Ready suspend
Ready
Short-term scheduling
Running
Exit
Medium-term Scheduling
Terminated
Event wait
Waiting suspend
Waiting

Comparison between scheduler.

S.N.	Long Term	Short Term	Medium Term
1	It is job scheduler	It is CPU Scheduler	It is swapping
2	Speed is less than short term scheduler	Speed is very fast	Speed is in between both
3	It controls degree of multiprogramming	Less control over degree of multiprogramming	Reduce the degree of multiprogramming.
4	Absent or minimal in time sharing system.	Minimal in time sharing system.	Time sharing system use medium term scheduler.
5	It select processes from pool and load them into memory for execution.	It select from among the processes that are ready to execute.	Process can be reintroduced into memory and its execution can be continued.
6	Process state is (New to Ready)	Process state is (Ready to Running)	swapping
7	Select a good process, mix of I/O bound and CPU bound.	Select a new process for a CPU quite frequently.	

CPU Scheduling

- Types: Preemptive and non preemptive
- Assumption initially: No process has I/O requirement
- Some terminologies:
  - Arrival Time(AT): Process arrival time(admitted by LTS)
  - Burst/Service time(BT): burst time is the time, required the cpu to execute that job, it is in milli seconds.
  - Completion time (CT): Time when process completes execution
  - Turn around time (TAT): Time from arrival until completion (TAT=CT-AT)
  - Waiting time (WT): Waiting for CPU in ready queue (WT=TAT-BT)
  - Response time: Time from arrival to the first response on CPU
  - Scheduling length: max(CT)-min(AT)
  - Throughput: No of processes executed per unit time

Scheduling Criteria

- CPU Utilization
- Throughput
- Turnaround time
- Waiting Time
- Response Time

CPU SCHEDULING ALGORITHMS:

- FCFS (First come first serve) – Non preemptive
- SJF (Shortest Job first) – Non preemptive
- Priority Scheduling
- Round Robin - Preemptive
- Others

FCFS (First Come First Serve)

- The process that request the CPU first is holds the cpu first.
- If a process request the cpu then it is loaded into the ready queue
- Mode: Non preemptive
- Tie breaker: Smallest ID first of the processes at tie (P1 and P2, P1 will execute first)

In FCFS Scheduling,

- The process which arrives first in the ready queue is firstly assigned the CPU.
- In case of a tie, process with smaller process id is executed first.
- It is always non-preemptive in nature.

Advantages-

It is simple and easy to understand.

- It can be easily implemented using queue data structure.
- It does not lead to starvation.

Disadvantages-

It does not consider the priority or burst time of the processes.

- It suffers from **convoy effect**.

Applications : billing counter system of shopping mall , buying tickets at ticket counter, Banking, Toll,

Process Id	Arrival time	Burst time
P1	3	4
P2	5	3
P3	0	2
P4	5	1
P5	4	3

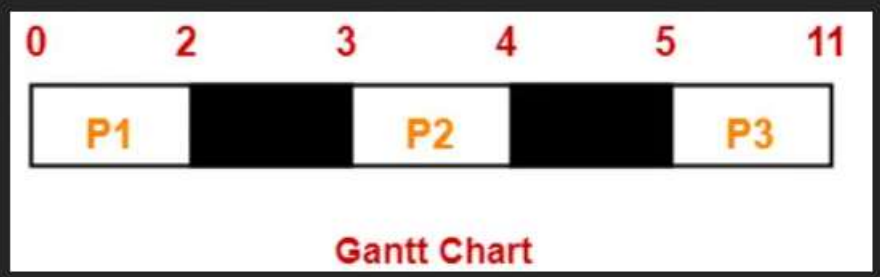
Process Id	Exit time	Turn Around time	Waiting time
P1	7	7 – 3 = 4	4 – 4 = 0
P2	13	13 – 5 = 8	8 – 3 = 5
P3	2	2 – 0 = 2	2 – 2 = 0
P4	14	14 – 5 = 9	9 – 1 = 8
P5	10	10 – 4 = 6	6 – 3 = 3

- Turn Around time = Exit time – Arrival time
- Waiting time = Turn Around time – Burst time

Now,

- Average Turn Around time = (4 + 8 + 2 + 9 + 6) / 5 = 29 / 5 = 5.8 unit
- Average waiting time = (0 + 5 + 0 + 8 + 3) / 5 = 16 / 5 = 3.2 unit

Process Id	Arrival time	Burst time
P1	0	2
P2	3	1
P3	5	6



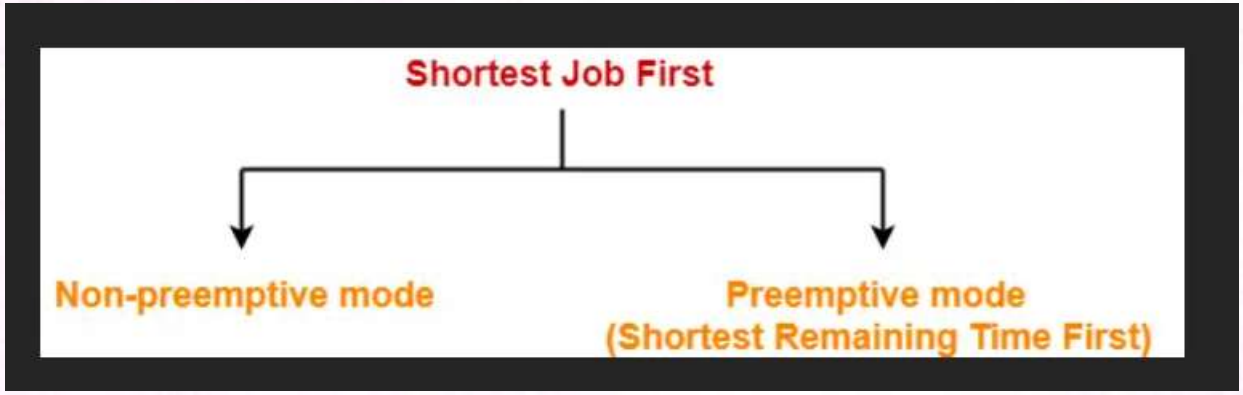
Process Id	Exit time	Turn Around time	Waiting time
P1	2	2 – 0 = 2	2 – 2 = 0
P2	4	4 – 3 = 1	1 – 1 = 0
P3	11	11- 5 = 6	6 – 6 = 0

Now,

- Average Turn Around time = (2 + 1 + 6) / 3 = 9 / 3 = 3 unit
- Average waiting time = (0 + 0 + 0) / 3 = 0 / 3 = 0 unit

In SJF Scheduling,

- Out of all the available processes, CPU is assigned to the process having smallest burst time.
- In case of a tie, it is broken by **FCFS Scheduling**.

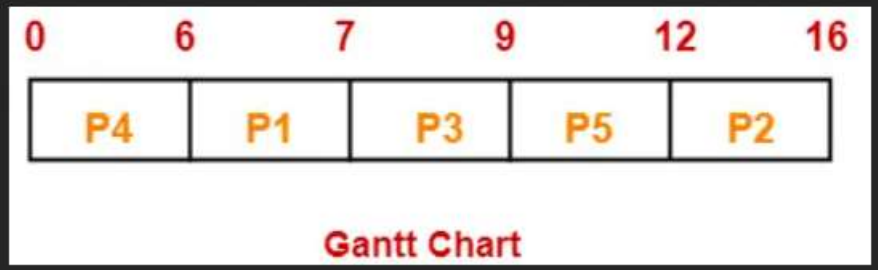


More about SJF

- Advantages:
  - Min waiting time among non preemptive scheduling
  - Better throughput in continuous execution
  - No convoy effect

- Disadvantages
  - It can not be implemented practically since burst time of the processes can not be known in advance.
  - It leads to starvation for processes with larger burst time.
  - Priorities can not be set for the processes.
  - Processes with larger burst time have poor response time.

Process Id	Arrival time	Burst time
P1	3	1
P2	1	4
P3	4	2
P4	0	6
P5	2	3



Gantt Chart

Process Id	Exit time	Turn Around time	Waiting time
P1	7	7 – 3 = 4	4 – 1 = 3
P2	16	16 – 1 = 15	15 – 4 = 11
P3	9	9 – 4 = 5	5 – 2 = 3
P4	6	6 – 0 = 6	6 – 6 = 0
P5	12	12 – 2 = 10	10 – 3 = 7

Now,

- Average Turn Around time = (4 + 15 + 5 + 6 + 10) / 5 = 40 / 5 = 8 unit
- Average waiting time = (3 + 11 + 3 + 0 + 7) / 5 = 24 / 5 = 4.8 unit

Process Id	Arrival time	Burst time
P1	3	1
P2	1	4
P3	4	2
P4	0	6
P5	2	3

Process Id	Exit time	Turn Around time	Waiting time
P1	4	4 – 3 = 1	1 – 1 = 0
P2	6	6 – 1 = 5	5 – 4 = 1
P3	8	8 – 4 = 4	4 – 2 = 2
P4	16	16 – 0 = 16	16 – 6 = 10
P5	11	11 – 2 = 9	9 – 3 = 6

Now,

- Average Turn Around time = (1 + 5 + 4 + 16 + 9) / 5 = 35 / 5 = 7 unit
- Average waiting time = (0 + 1 + 2 + 10 + 6) / 5 = 19 / 5 = 3.8 unit

Process Id	Arrival time	Burst time
P1	0	9
P2	1	4
P3	2	9

If the CPU scheduling policy is SRTF, calculate the average waiting time and average turn around time.

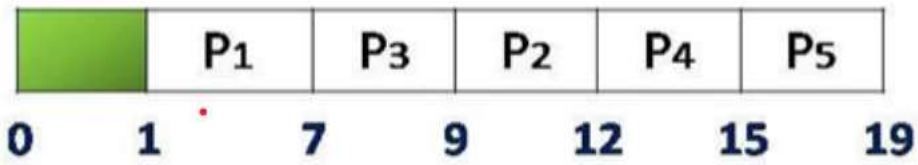
Solution-

Gantt Chart-

Example

Process No.	Arrival Time (AT)	Burst Time (BT)
1	1	6
2	2	3
3	3	2
4	4	3
5	5	4

Gantt chart:



TAT = CT - AT and WT = TAT - BT

Process No.	AT	BT	CT	TAT	WT
1	1	6	7	(7 – 1) = 6	(6 – 6) = 0
2	2	3	12	(12 – 2) = 10	(10 – 3) = 7
3	3	2	9	(9 – 3) = 6	(6 – 2) = 4
4	4	3	15	(15 – 4) = 11	(11 – 3) = 8
5	5	4	19	(19 – 5) = 14	(14 – 4) = 10

Shortest Remaining Time First (SRTF) (Preemptive SJF)

Process No.	Arrival Time (AT)	Burst Time (BT)
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5

Gantt Chart

TAT = CT - AT, WT = TAT – BT

And

Response Time (RT) = FR (First Response) – AR (Arrival Time)

P_No.	AT	BT	CT	TAT	WT	First Response (FR)	RT
P1	0	8	17	(17 – 0) = 17	(17 – 8) = 9	0	(0 – 0) = 0
P2	1	4	5	(5 – 1) = 4	(4 – 4) = 0	1	(1 – 1) = 0
P3	2	9	26	(26 – 2) = 24	(24 – 9) = 15	17	(17 – 2) = 15
P4	3	5	10	(10 – 3) = 7	(7 – 5) = 2	5	(5 – 3) = 2

Example: (GATE: 2007)

Process No.	Arrival Time (AT)	Burst Time (BT)
P_1	0	20
P_2	15	25
P_3	30	10
P_4	45	15

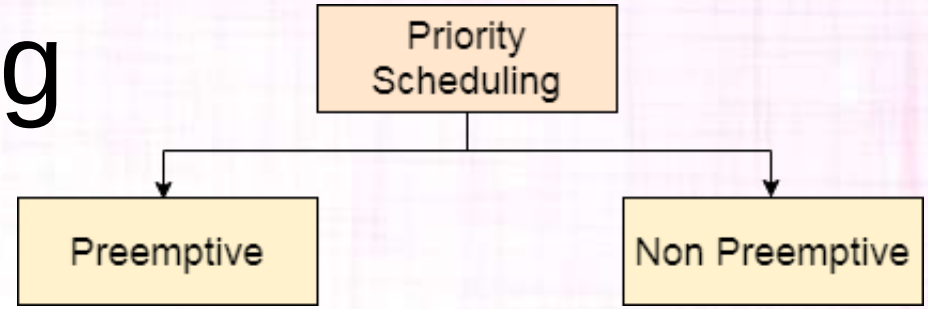
Gantt Chart as per **SRTF**.

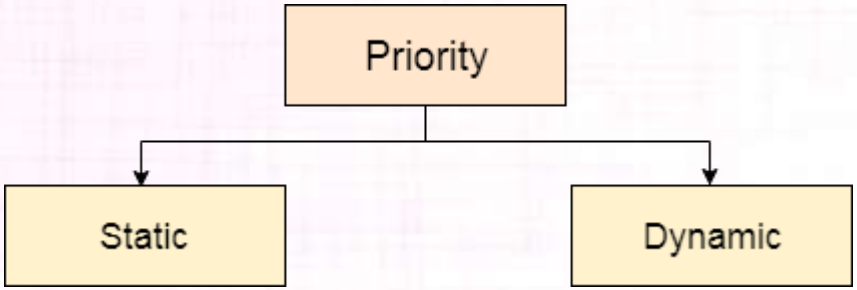
P_1	P_1	P_2	P_3	P_2	P_2	P_4

0 15 20 30 40 45 55 70

Process No.	AT	BT	CT	TAT	WT
1	0	~~20~~ 0	20	20	0
2	15	~~25~~ ~~15~~ 10	55	40	15
3	30	10	40	10	0
4	45	15	70	25	10

Priority Scheduling





Process No.	Priority	Arrival Time (AT)	Burst Time (BT)
P_1	2	0	2
P_2	4	1	5
P_3	6	2	1
P_4	10	3	2
P_5	8	4	3
P_6	12	5	6

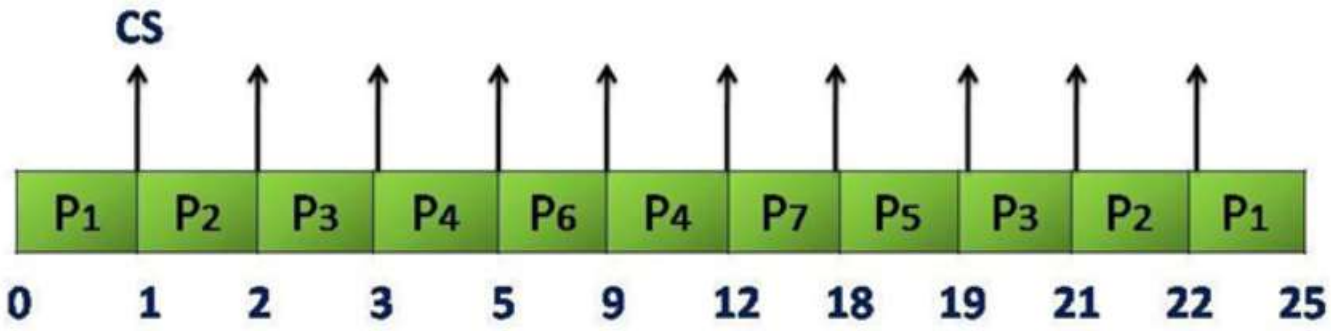
Gantt Chart:

P_No.	Priority	AT	BT	CT	TAT	WT	First Response (FR)	RT
1	2	0	2	2	(2 – 0) = 2	(2 – 2) = 0	0	(0 – 0) = 0
2	4	1	5	19	(19 – 1) = 18	(18 – 5) = 13	14	(14 – 1) = 13
3	6	2	1	3	(3 – 2) = 1	(1 – 1) = 0	2	(2 – 2) = 0
4	10	3	2	5	(5 – 3) = 2	(2 – 2) = 0	3	(3 – 3) = 0
5	8	4	3	14	(14 – 4) = 10	(10 – 3) = 7	11	(11 – 4) = 7
6	12	5	6	11	(11 – 5) = 6	(6 – 6) = 0	5	(5 – 5) = 0

- **Step-1:** Select the first process whose arrival time will be 0, we need to select that process because that process is only executing at time t=0.
- **Step-2:** Check the priority of the next available process. Here we need to check for 3 conditions.
    - if $\textbf{priority}_{(current_process)} > \textbf{priority}_{(prior_process)}$:- then execute the current process.
    - if $\textbf{priority}_{(current_process)} < \textbf{priority}_{(prior_process)}$:- then execute the prior process.
    - if $\textbf{priority}_{(current_process)} = \textbf{priority}_{(prior_process)}$:- then execute the process which arrives first i.e., arrival time should be first.
- **Step-3:** Repeat **Step-2** until it reaches the final process.
- **Step-4:** When it reaches the final process, choose the process which is having the highest priority & execute it. Repeat the same step until all processes complete their execution.

Process No.	Priority	Arrival Time (AT)	Burst Time (BT)
P_1	2	0	4
P_2	4	1	2
P_3	6	2	3
P_4	10	3	5
P_5	8	4	1
P_6	12	5	4
P_7	9	6	6

Gantt Chart:



P_No.	Priority	AT	BT	CT	TAT	WT	First Response (FR)	RT
1	2	0	4	25	(25 – 0) = 25	(25 – 4) = 21	0	(0 – 0) = 0
2	4	1	2	22	(22 – 1) = 21	(21 – 2) = 19	1	(1 – 1) = 0
3	6	2	3	21	(21 – 2) = 19	(19 – 3) = 16	2	(2 – 2) = 0
4	10	3	5	12	(12 – 3) = 9	(9 – 5) = 4	3	(3 – 3) = 0
5	8	4	1	19	(19 – 4) = 15	(15 – 1) = 14	18	(18 – 4) = 14
6	12	5	4	9	(9 – 5) = 4	(4 – 4) = 0	5	(5 - 5) = 0
7	9	6	6	18	(18 – 6) = 12	(12 – 6) = 6	12	(12 – 6) = 6

P1	P3	P6	P4	P2	P5	P7

0 3 7 11 13 18 27 37

1.Turn Around Time = Completion Time - Arrival Time
2.Waiting Time = Turn Around Time - Burst Time

Process Id	Priority	Arrival Time	Burst Time	Completion Time	Turnaround Time	Waiting Time	Response Time
1	2	0	3	3	3	0	0
2	6	2	5	18	16	11	13
3	3	1	4	7	6	2	3
4	5	4	2	13	9	7	11
5	7	6	9	27	21	12	18
6	4	5	4	11	6	2	7
7	10	7	10	37	30	18	27

Avg Waiting Time = (0+11+2+7+12+2+18)/7 = 52/7 units

Preemptive Priority Scheduling

Process Id	Priority	Arrival Time	Burst Time
1	2(L)	0	1
2	6	1	7
3	3	2	3
4	5	3	6
5	4	4	5
6	10(H)	5	15
7	9	15	8

P1	P2	P3	P5	P4	P2	P7	P6

0 1 2 5 10 16 22 30 45

Process Id	Priority	Arrival Time	Burst Time	Completion Time	Turn around Time	Waiting Time
1	2	0	1	1	1	0
2	6	1	7	22	21	14
3	3	2	3	5	3	0
4	5	3	6	16	13	7
5	4	4	5	10	6	1
6	10	5	15	45	40	25
7	9	6	8	30	24	16

Round Robin Scheduling

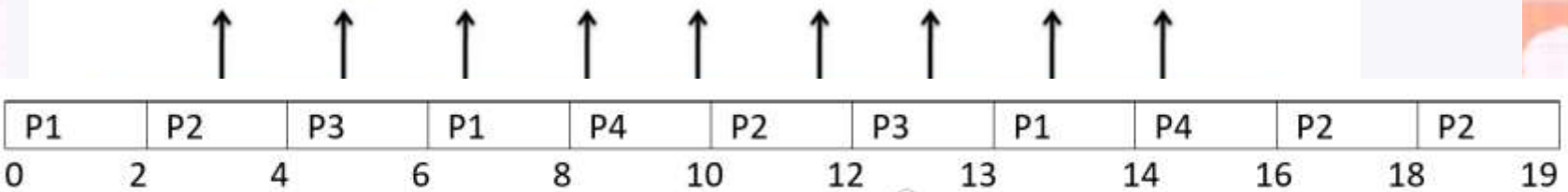
- Criteria: Arrival time + (Q)Quantum time/timeslice
- Mode: Preemptive
- Quantum time/time slice: Max amount of time for which a process runs on CPU before preemption

Example 1: TQ = 2 (Here)

Process No.	Arrival Time (AT)	Burst Time (BT)
P_1	0	5
P_2	1	7
P_3	2	3
P_4	3	4

Gantt Chart

Context Switching



Now we calculate **Turn Around Time (TAT)** and **Waiting Time (WT)** using the following formula:

TAT = CT - AT, WT = TAT – BT

And

Response Time (RT) = FR (First Response) - AR (Arrival Time)

P_No.	AT	BT	CT	TAT	WT	First Response (FR)	Response Time (RT)
P1	0	~~5~~ ~~3~~ ~~1~~	16	(16 – 0) = 16	(16 – 5) = 11	0	(0 – 0) = 0
P2	1	7 ~~5~~ ~~3~~	19	(19 – 1) = 18	(18 – 7) = 11	2	(2 – 1) = 1
P3	2	3 ~~2~~ ~~1~~	13	(13 – 2) = 11	(11 – 3) = 8	4	(4 – 2) = 2
P4	3	4 ~~2~~ ~~2~~	15	(15 – 3) = 12	(12 – 4) = 8	6	(6 – 3) = 3

This Photo by Unknown Author is licensed under CC BY